Práctica 4: guía de arranque por roles

Árboles de decisión y control de la complejidad. Sesión en clase: martes 17 de noviembre. Entrega: lunes 23 de noviembre de 2026, antes de clase, una por equipo en Moodle.

REGLA Nº 1: NO SE TOCA EL TRABAJO DE OTROS. Puedes abrirlo, copiarlo a tu notebook y usarlo, pero **no cambias ni borras** sus archivos. Si ves un fallo, se lo dices a su dueño y lo cambia él. Quien junta los notebooks solo adapta lo imprescindible, y cualquier cambio de código o de texto lo consulta antes con su dueño.

Quién hace qué en la P4

Rol	Persona	Parte principal
R1 Datos	Fernando	Representación para árboles (sin escalado, one-hot) en los dos retos y `features.md`
R2 Clasificación	Marina	P4.1, P4.2 e importancias de la P4.4 (cyber), parte de cyber de la P4.5
R3 Regresión	Diana	P4.3 y reglas de la P4.4 (storage), parte de storage de la P4.5
R4 Evaluación	Francisco	Folds para las rejillas, métricas, `comparativa.csv`, diferencias por fold (P4.5) y revisión de penalizaciones
R5 Informe	Claudia	Notebooks finales, `roles.md`, `AI_USAGE.md`, ejecución limpia y entrega

Propuesta de reparto de la P4.4: el enunciado deja sacar las 2 reglas de cualquier reto. Si las saca R3 del árbol de storage, R2 y R3 quedan más equilibrados (R2 ya tiene P4.1 y P4.2). Las importancias tienen que ser de cyber, así que esas son de R2.

Reglas para todos

- **Todo lo entregable en inglés.** Las interpretaciones van en celdas Markdown: en un comentario de código no puntúan.
- Primera celda de cada notebook: `PROJECT_RANDOM_STATE = 2627` y un print. Rutas siempre relativas, nunca `C:\...`.
- **Solo cuenta lo que aparece en las salidas guardadas.** Las gráficas también: comprobad que se ven en el notebook guardado. Cada número que escribáis en un texto tiene que salir en alguna salida (en la P3 se nos coló uno).
- Los folds se **leen** de `folds/`, no se reconstruyen. Toda rejilla o curva usa `cv=folds`, nunca `cv=5`. `feature_group`, `config_group`, `fold_id` y `row_id` nunca entran al modelo.
- El encoder siempre dentro de un `Pipeline`. La variación entre folds se da como **rango (máx − mín)**, como en la P3. Nunca se dice que una diferencia es “significativa”.
- Cada uno apunta su uso de IA y se lo pasa a R5.

Cómo trabajamos en GitHub

- **La rama va con el rol, no con la persona:** `r1-datos` (Fernando), `r2-cyber` (Marina), `r3-storage` (Diana), `r4-evaluacion` (Francisco) y `r5-informe` (Claudia).
- Antes de empezar: `git fetch`, `git checkout tu-rama` y `git merge origin/main`. Las ramas ya están limpias: lo de la P3 está guardado en `entrega_p3/`.
- **Tu plantilla ya está en** `notebooks/`: `R1_p4.ipynb`, `R2_p4_cyber.ipynb`, `R3_p4_storage.ipynb` y `R4_p4.ipynb`. Las rutas son `../datos/...` y `../folds/...`.
- Se sube con pull request a `main`. Antes de abrirlo, mira en “Files changed” que no borras ni cambias archivos de otros.

Orden de trabajo y fechas (propuesta)

Cuándo	Qué
Vie 13 - dom 15	R1 publica la representación de los dos retos. R4 publica los folds (`cv=folds`) y las métricas.
Lun 16 - jue 19	R2 y R3 hacen sus rejillas, modelos, gráficas y reglas. **Martes 17, sesión en clase:** dudas al profe.
Vie 20 - sáb 21	R4 añade las filas a la tabla y las diferencias por fold. R2 y R3 responden las 6 preguntas. R5 junta y ejecuta.

Cuándo	Qué
Dom 22	Todos revisan los notebooks finales y su entrada de IA.
Lun 23, antes de clase	Entrega en Moodle.

R1 · Datos y representación — Fernando

Te encargas de: la representación de los dos retos para el árbol y `features.md`.

Tus archivos: `notebooks/R1_p4.ipynb` y `plantillas/features.md`.

Cómo empezar

- **1. Sin escalado, y decir por qué:** el árbol corta por umbrales, y escalar solo mueve el umbral sin cambiar qué filas van a cada lado (notebook del tema 8, sección 13). Pasa lo mismo con `log1p` en cyber. Propuesta: quitar los dos, porque así los umbrales salen directamente en bytes y las reglas se leen más fácil. Si dejáis `log1p`, R2 tendrá que traducir cada umbral con $e^x - 1$.
- **2. One-hot sin quitar ningún nivel:** `OneHotEncoder(handle_unknown="ignore", sparse_output=False)`. Un árbol no necesita nivel de referencia, y quitarlo solo esconde ese nivel. Sale con 33 columnas en cyber y 17 en storage (probado).
- **3. Escribir la correspondencia con la P3**, porque la P4.5 la necesita. La P3 quitaba un nivel (`tcp`, `-`, `SF`; `random`, `write`, `4+1`, `nvme`) y escalaba. La P4 deja todos los niveles y no escala.
- **4. Nivel no visto:** con `"ignore"` sale todo a 0. Ahora ya no equivale a un nivel de referencia: es "ninguno de los servicios conocidos". Pasa de verdad: `service=gssapi` no aparece en el train del fold 3.
- **5.** `make_preprocessor()` para cada reto: `ColumnTransformer([("cat", OneHotEncoder(...), NOMINAL), ("num", "passthrough", NUMERIC)])`, con una comprobación dentro de cada fold.
- **6. Ampliar** `features.md` con una entrada fechada de la P4: sin variables nuevas, solo el cambio de representación (sin escalado y one-hot completo). Ruta `primary`.

Entregas

Celdas de preprocesado listas para copiar, explicación en Markdown y `features.md` ampliado.

Dependes de / te esperan

No dependes de nadie, así que **empieza primero**. R2, R3 y R4 esperan tus bloques.

R2 · Clasificación (reto A, cyber) — Marina

Te encargas de: P4.1, P4.2 y las importancias de la P4.4 (5,5 de los 10 puntos) y la parte de cyber de la P4.5.

Tus archivos: `notebooks/R2_p4_cyber.ipynb`.

Cómo empezar

- **1. Modelo:** `DecisionTreeClassifier(random_state=PROJECT_RANDOM_STATE)` en un `Pipeline` con el bloque de R1.
- **2. Declarar la rejilla y por qué.** Ejemplo: `max_depth` [5, 8, 12, 16, 20, None], `min_samples_leaf` [1, 5, 20, 50], `ccp_alpha` [0, 1e-5, 1e-4, 1e-3] y `criterion` [gini, entropy]. Un árbol completo tarda ~1,2 s por fold, así que la rejilla entera son pocos minutos.
- **3.** `GridSearchCV(pipe, grid, cv=folds, scoring="f1_macro", return_train_score=True)`. Gana la mejor media de macro F1. **Sin partición aparte.**
- **4. Tamaño del árbol:** `tree_.node_count`, `get_depth()`, `get_n_leaves()` y el primer corte (variable y umbral), diciendo de qué ajuste salen. Ojo: `best_estimator_` está ajustado con todas las filas, así que es una descripción, no una evaluación. Como referencia, el árbol sin límites del fold 0 tiene 2.437 nodos, profundidad 42 y 1.219 hojas.
- **5. Métricas como siempre:** macro F1 por fold con su media y su rango, weighted F1, accuracy, tabla por clase con support y matriz de confusión con nombres.
- **6. P4.2, curvas de train y validación juntas** para 2 hiperparámetros (por ejemplo `max_depth` y `ccp_alpha`). Probado en el fold 0: con profundidad 3, 0,46 en train y 0,46 en validación (se queda corto); sin límite, 0,997 en train y 0,935 en validación (se pasa). Hay que poner en cada gráfica qué lado es cada cosa.
- **7. Ruta de poda** en el train de un fold que nombréis, pasándole las filas ya transformadas (`pipe[:-1].fit_transform(...)`). En el fold 0 salen 621 alphas, de casi 0 a 0,092 (en unidades de Gini).
- **8. Importancias** del árbol elegido, diciendo de qué ajuste salen. Se suman las columnas one-hot para volver a las 11 variables originales, se hace la gráfica y se escriben las 2 frases obligatorias: qué dice una importancia y qué no dice.

- **9. P4.5:** con las diferencias que publica R4, las 6 preguntas contra la logística de la P3 y contra el KNN de la P2.

Entregas

Secciones P4.1, P4.2 y las importancias de la P4.4, y las 6 preguntas de cyber de la P4.5.

Dependes de / te esperan

Necesitas el bloque de R1 y los folds y métricas de R4. R4 espera tus 5 valores de macro F1 y el resto de datos de tu fila.

R3 · Regresión (reto B, storage) — Diana

Te encargas de: P4.3 y las reglas de la P4.4 (sacadas del árbol de storage) y la parte de storage de la P4.5.

Tus archivos: `notebooks/R3_p4_storage.ipynb`.

Cómo empezar

- **1. Modelo:** `DecisionTreeRegressor(random_state=PROJECT_RANDOM_STATE)` sobre `z = log10(latency_ns)`, en un `Pipeline` con el bloque de R1.
- **2. Rejilla propia, pensada para estos datos**, no copiada de cyber. Ejemplo: `max_depth` [4, 6, 8, 10, 12, None], `min_samples_leaf` [1, 5, 10, 25, 50] y `ccp_alpha` [0, 1e-5, 1e-4, 1e-3]. Va rápido: un árbol completo tarda 0,05 s.
- **3.** `GridSearchCV(pipe, grid, cv=folds, scoring="neg_mean_absolute_error", return_train_score=True)`. sklearn devuelve el MAE con el signo cambiado.
- **4. Dato para la interpretación:** el árbol sin límites del fold 0 tiene 1.950 hojas, justo el número de configuraciones del train. Hace una hoja por configuración y promedia sus 5 réplicas. Aun así saca MAE(z) 0,126, contra 0,180 del KNN y 0,198 de la lineal.
- **5. Métricas:** MAE(z) por fold con su media y su rango, RMSE(z) y R2(z) sobre z, y MAE en ns **después** de `10 ** z_hat`.
- **6. Pregunta con nota (1 punto, 0,25 cada parte):** qué habría ido mal copiando los hiperparámetros de cyber. Hay que hablar de 4 cosas: el tamaño (34.928 filas frente a 12.190 y 2.438 configuraciones), las réplicas de 5 (un `min_samples_leaf` de 5 equivale a una configuración), la impureza (Gini frente a error cuadrático) y las unidades de `ccp_alpha` (la ruta de cyber llega a 0,09 en Gini y la de storage a 0,43 en z^2).
- **7. Reglas de 2 hojas con nombre:** nodo, valor predicho (z y 10^z ns) y filas de train de cada una, con todas las condiciones desde la raíz (`children_left`, `children_right`, `feature` y `threshold`). Hay que decir de qué árbol salen: el elegido o uno menos profundo solo para leer, marcado así.
- **8. Una regla leída como frase de ingeniero de storage:** `device_type_hdd <= 0.5` quiere decir “el disco no es hdd”, y la latencia de la hoja es 10^z ns.
- **9. P4.5:** con las diferencias que publica R4, las 6 preguntas contra la lineal de la P3 y contra el KNN de la P2.

Entregas

Sección P4.3, las reglas de la P4.4 y las 6 preguntas de storage de la P4.5.

Dependes de / te esperan

Necesitas el bloque de R1 y los folds y métricas de R4. R4 espera tus 5 valores de MAE(z) y el resto de datos de tu fila.

R4 · Evaluación y validación — Francisco

Te encargas de: los folds para las rejillas, las métricas, la tabla comparativa, las diferencias por fold y la revisión de penalizaciones. Puedes vetar cualquier evaluación que se salte las reglas.

Tus archivos: `notebooks/R4_p4.ipynb` y `comparativa.csv`.

Cómo empezar

- **1. Publicar los folds de los dos retos** como lista de `(train_idx, val_idx)` leída de `folds/`, lista para `cv=folds`. Se reutilizan `canonical_cv` y `check_folds` de la P3.
- **2. Publicar las métricas** de los dos retos, como en la P3. La variación se da como rango (máx − mín).
- **3. Leer de** `comparativa.csv` **los scores por fold de la P2 y la P3**, sin copiarlos a mano (es lo que nos señaló Fernando en la P3).
- **4. Diferencias por fold, 4 grupos:** el árbol contra la P3 y contra la P2 en cada reto. Macro F1: `delta = árbol - anterior`. MAE(z): `delta = anterior - árbol`. Los 5 valores, su media y su rango, leídos de forma descriptiva.

- **5. Tipo de comparación:** el árbol no escala y usa el one-hot completo. La P3 quitaba un nivel y escalaba, y la P2 escalaba. Las dos son **comparaciones de pipelines**. Además, escalar no cambia nada en un árbol, y el enunciado quiere que se diga.
- **6. Añadir las 2 filas de la P4** a `comparativa.csv` con la columna `route`. Las filas de la P2 y la P3 no se tocan.
- **7. Revisar los notebooks de R2 y R3** con la lista de penalizaciones: `cv=folds` en toda rejilla y curva, encoder dentro del fold, hiperparámetros no copiados de cyber a storage, regla log a físico, nada de "significativo" y nada de comparar semillas.

Entregas

Código de folds y métricas, `comparativa.csv` ampliado, las diferencias de los dos retos y la revisión.

Dependes de / te esperan

Empiezas a la vez que R1. Para la tabla necesitas los números de R2 y R3. Las 6 preguntas las escriben ellos y tú las revisas.

R5 · Informe y reproducibilidad — Claudia

Te encargas de: los notebooks finales, los archivos del equipo, la ejecución limpia y la entrega.

Tus archivos: `notebooks/p4_cyber.ipynb`, `notebooks/p4_storage.ipynb`, `plantillas/roles.md` y `plantillas/AI_USAGE.md`.

Cómo empezar

- **1. Juntar por reto.** `p4_cyber`: R1 (cyber), R4 (cyber) y R2, con la P4.1, la P4.2, la P4.4 (importancias) y la P4.5. `p4_storage`: R1 (storage), R4 (storage) y R3, con la P4.3, las reglas de la P4.4 y la P4.5. Sin cambiar código ni interpretaciones de nadie sin consultarlo.
- **2. Actualizar** `roles.md` con la fila de la P4 marcada, y `AI_USAGE.md` con la sección de la P4, usando lo que os pase cada uno.
- **3. Probar en una copia limpia del repo:** reiniciar y ejecutar los dos notebooks enteros y guardar las salidas. Las rejillas tardan unos minutos. **Comprobar que todas las gráficas se ven en el notebook guardado**, porque el enunciado lo pide expresamente. Revisar también la semilla en la primera celda, el inglés y las rutas.
- **4. Entregar en Moodle:** `p4_cyber.ipynb`, `p4_storage.ipynb`, `comparativa.csv`, `features.md`, `roles.md` y `AI_USAGE.md`.
- **5. Después de entregar,** guardar la entrega en `entrega_p4/`, igual que Fernando hizo con la P3.

Entregas

Los 2 notebooks finales ejecutados, los archivos del equipo y la entrega hecha.

Dependes de / te esperan

Lo tuyo llega al final. Marca fechas a los demás para tener margen antes del lunes 23.

Ojo: la defensa es el **10 de diciembre** y entra todo, de la P2 a la P5. El 60 % es individual y te pueden preguntar por cualquier parte. Que cada uno lea y entienda lo de los demás (leer sí, tocar no).